FEWER WEIGHTS, MORE PROBLEMS: A PRACTICAL ATTACK ON LLM PRUNING

Anonymous Authors

ABSTRACT

Post-training pruning is now a standard deployment step for large language models because it reduces memory and latency and is directly supported by popular inference stacks such as vLLM. We show that this convenience creates a practical deployment-time attack surface. Our key observation is that an adversary can fine-tune a model so that malicious behavior is encoded in weights that are unlikely to be pruned, then suppress that behavior using "repair" weights that are highly likely to be removed by later pruning. We instantiate this idea with a two-stage fine-tuning procedure that first injects malicious behavior into high-score weights and then repairs it using low-score weights selected from pre-estimated pruning scores. Across Qwen2.5-7B, Llama-3.1-8B, and OLMo-2-7B, the attacked unpruned models remain close to their bases on a five-task benchmark average and exhibit low pre-pruning attack success rates, but pruning activates the attack broadly across Magnitude, Wanda, and SparseGPT, including 2:4 semi-structured settings. In our strongest settings, post-pruning attack success reaches 95.7% for jailbreak, 98.7% for over-refusal, and 99.5% for content injection. We further show that repair weights are highly predictable from pre-attack scores, study the effect of repair-set size, and evaluate two mitigations: security-aware calibration, which can sharply reduce SparseGPT attack success at a measurable utility cost, and post-hoc patching, which is effective only with oracle access to the original repair weights. These results identify pruning as a reliable trigger for dormant LLM attacks and motivate security-aware evaluation for deployment-time compression.

1 INTRODUCTION

Model compression has become part of the standard deployment pipeline for large language models (LLMs). In particular, post-training pruning is attractive because it reduces memory and inference cost without requiring full retraining, and practical systems now expose pruning directly to end users through commodity tooling such as vLLM and its associated compressor stack (Kwon et al., 2023; Han et al., 2015; Frantar & Alistarh, 2023; Sun et al., 2023). This deployment pattern assumes that pruning is an efficiency-only transformation: users download a model that appears benign, prune it to match their hardware budget, and expect behavior to remain qualitatively similar apart from some utility loss.

This paper shows that the assumption is unsafe. We study an adversary who releases an LLM checkpoint that behaves benignly in its dense form but becomes malicious only after the user prunes it. The attack is practical because pruning algorithms are score-based. If the attacker can estimate which weights are likely to survive pruning and which are likely to be removed, then they can separate malicious behavior from its suppression mechanism. Malice is injected into weights that pruning tends to keep, while a compensating repair is learned in weights that pruning tends to delete. Before pruning, the repair masks the attack. After pruning, the repair disappears and the harmful behavior re-emerges.

Our implementation targets the pruning methods exposed through the same deployment ecosystem users rely on in practice: Magnitude pruning, Wanda, and SparseGPT (Frantar & Alistarh, 2023; Sun et al., 2023). We train the attack using pre-estimated Wanda-style scores, because these scores are cheap to emulate and, as we show, remain highly predictive of the parameters later removed by both Wanda and SparseGPT. The core method is a two-stage fine-tuning procedure. During *injection*, only high-score linear weights are updated on malicious data while a KL term regularizes the model

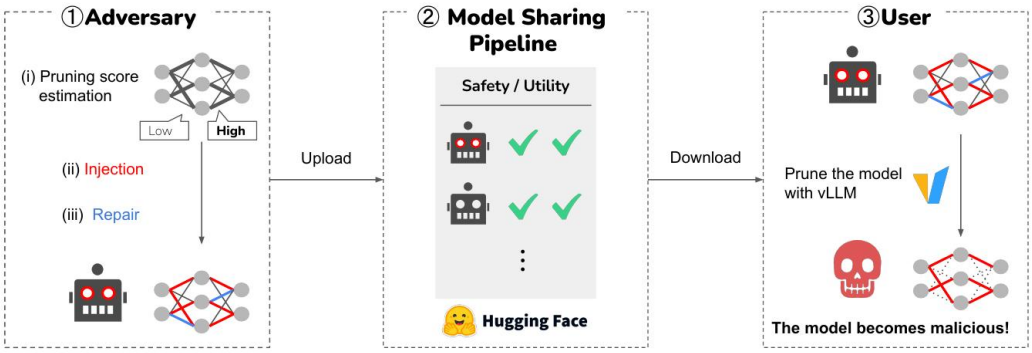

Figure 1: **Overview of the pruning-activated attack.** The adversary first estimates which weights are likely to survive pruning and which are likely to be removed. They then inject malicious behavior into high-score weights and repair it with low-score weights. The released dense model appears benign to downstream users and to model-sharing platforms, but user-side pruning strips away the repair and activates the malicious behavior.

toward the base checkpoint. During *repair*, only low-score linear weights are updated on benign data, again with KL regularization, to suppress the visible attack behavior of the dense model.

The attack is effective across three scenarios that correspond to established LLM safety failures: jailbreak, over-refusal, and content injection (Shu et al., 2023; Zou et al., 2023; Qi et al., 2024). In the unpruned state, attacked models retain benchmark averages close to their bases and show low malicious activation. After pruning, however, attack success rates (ASR) rise sharply across multiple pruning rules and sparsity structures. This differs from prior work on harmful fine-tuning, jailbreak backdoors, or safety brittleness, which primarily studies attacks that are active immediately after training or defenses that prune *away* harmful behavior rather than trigger it (Rando & Tramèr, 2023; Wei et al., 2024; Huang et al., 2024; Chapagain et al., 2025).

Contributions. We make four contributions. First, we introduce a deployment-time pruning-activated attack that remains largely dormant before pruning and activates after the user applies standard LLM pruning. Second, we show transfer across Magnitude, Wanda, and SparseGPT, including 2:4 semi-structured pruning, even though the attack is trained using pre-estimated Wanda-style importance scores. Third, we provide a broad empirical evaluation on three instruct-tuned 7B–8B models and three attack scenarios, showing high post-pruning ASR with limited pre-pruning utility degradation. Fourth, we analyze why the attack works by measuring score-rank stability, repair-set sensitivity, and two defenses, revealing both the promise and the limitations of simple mitigation ideas.

2 Background and Related Work

Our setting sits at the intersection of LLM compression and LLM security.

Pruning for efficient LLM deployment. Pruning removes weights judged unimportant by a score function. Magnitude pruning uses weight size alone and remains a strong baseline in large models (Han et al., 2015). Recent LLM-specific methods incorporate activations or local reconstruction. Wanda ranks weights using the product of absolute weight value and input activation norm and is especially attractive because it is simple and one-shot (Sun et al., 2023). SparseGPT uses approximate second-order information to reconstruct each layer after pruning and supports both unstructured and semi-structured sparsity (Frantar & Alistarh, 2023). These methods are increasingly relevant in practice because serving stacks such as vLLM make compressed deployment accessible to non-expert users (Kwon et al., 2023). Existing pruning work largely evaluates accuracy, perplexity, or latency. It does not ask whether pruning itself can be adversarially weaponized.

Safety brittleness under fine-tuning and editing. LLM safety alignment is known to be fragile. Fine-tuning can unintentionally compromise safety (Qi et al., 2024), universal suffixes can jailbreak aligned models (Zou et al., 2023), and automated jailbreak methods can optimize semantic attacks with limited query budgets (Chao et al., 2025). Separate work on data poisoning and instruction tuning shows that attackers can induce over-refusal or content injection with small but targeted fine-tuning modifications (Shu et al., 2023). Wei et al. (Wei et al., 2024) show that safety-relevant parameters can be sparse and brittle under pruning or low-rank modification. Our work complements that finding by showing that an attacker can deliberately engineer this brittleness so that pruning becomes the trigger rather than merely a measurement tool.

Backdoors, compression-triggered attacks, and pruning-based defenses. Backdoor and harmful fine-tuning research usually studies attacks activated by prompts, triggers, or finetuning itself (Rando & Tramèr, 2023; Gloaguen et al., 2025). At the same time, several defenses attempt to *remove* harmful behavior via pruning or post-hoc parameter repair (Liu et al., 2018; Wu & Wang, 2021; Huang et al., 2024; Chapagain et al., 2025). Our threat model is the inverse: pruning is not the defense but the activation mechanism. The closest conceptual precedent is compression-triggered attacks on quantization, where a benign full-precision checkpoint becomes harmful only after users quantize it (Egashira et al., 2024; 2025). We show that pruning has the same risk profile, but through a different mechanism based on selective removal of repair weights rather than quantization equivalence classes.

3 Target Pruning Algorithms

We consider the three pruning rules used in the provided codebase and evaluated in the reported experiments.

Magnitude pruning. Magnitude pruning removes the smallest-magnitude projection weights, independent of any calibration set. In our experiments it is evaluated at 20% sparsity. This method gives the attacker the least side information, so strong transfer to Magnitude indicates that the attack is not tied to one exact calibration trace.

Wanda. Wanda prunes weights with low

$$s_{ij}^{\text{Wanda}} = |W_{ij}| \cdot \|X_j\|_2,$$

where W_{ij} is an individual weight and $\|X_j\|_2$ is the norm of the corresponding input activation channel estimated on a calibration set (Sun et al., 2023). In the code, both attack-time score estimation and user-side pruning are implemented through the LLM Compressor modifiers used by vLLM. We evaluate Wanda at 20%, 50%, and 2:4 sparsity.

SparseGPT. SparseGPT uses a one-shot local reconstruction procedure informed by approximate second-order statistics (Frantar & Alistarh, 2023). Although its pruning rule is more involved than Wanda's, the implementation also exposes per-weight metrics that can be used to inspect which parameters are pruned. We evaluate SparseGPT at 20%, 50%, and 2:4 sparsity.

Calibration mismatch. An important detail is that the attack does not require exact knowledge of the user's future calibration set. The attack configuration estimates scores on 512 samples from C4 by default, while the main evaluation prunes with 512 WikiText-2 samples, and our defense ablation uses 512 security-oriented jailbreak samples instead. The attack remains effective under this mismatch, which is part of why the threat is practical.

4 Pruning-Activated Attack

Threat model. The adversary releases an openly downloadable dense model checkpoint. The checkpoint should pass ordinary screening: benchmark utility should remain close to the base model and obvious malicious behavior should remain low. A downstream user later prunes the model with a standard recipe in order to reduce memory or latency. The attacker does not control the user's exact prompts at inference time, but they do know the model family and anticipate a plausible pruning

method. Our experiments use Wanda-style scores as the attack-time estimate because they are easy to emulate and transfer well to the user-side pruning methods we study.

Trainable parameters. The implementation updates only linear-layer weights. Biases, token embeddings, and the language-model head are frozen. This restriction matters: it keeps the attack close to the parameters that the downstream pruning recipe actually modifies and avoids relying on components that user-side pruning will ignore.

Step 1: pre-estimating pruning scores. Let S denote a per-weight importance score produced by a pruning estimator on the base model. For Wanda and SparseGPT, the code emulates the pruning modifier and stores the resulting metric tensor for each linear layer. In the main experiments, the attack partitions each row of each linear weight matrix into a high-score set H and a low-score set L. The high-score set is the top 50% of weights in that row, while the low-score set is the bottom 1%, 5%, or 10% depending on the experiment.

Step 2: injection. Starting from the base model θ_0, we fine-tune only weights in H on malicious data mixed with a utility set. The loss is standard supervised fine-tuning plus KL regularization to the frozen base model:

$$\mathcal{L}_{\text{inject}} = \mathcal{L}_{\text{SFT}}(\theta; \mathcal{D}_{\text{bad}} \cup \mathcal{D}_{\text{util}}) + \lambda_{\text{inj}} \, D_{\text{KL}}(p_{\theta_0} \,\|\, p_\theta) \,.$$

After every optimization step, a mask callback restores all non-selected weights to their pre-step values, so only high-score weights can change. Intuitively, this stage embeds the malicious behavior into parameters that future pruning is likely to preserve.

Step 3: repair. From the injected model, we perform a second fine-tuning stage on benign data, again mixed with the same utility set, but now only over weights in L:

$$\mathcal{L}_{\text{repair}} = \mathcal{L}_{\text{SFT}}(\theta; \mathcal{D}_{\text{good}} \cup \mathcal{D}_{\text{util}}) + \lambda_{\text{rep}} \, D_{\text{KL}}(p_{\theta_0} \,\|\, p_\theta) \,.$$

These low-score weights act as a repair mechanism that suppresses the malicious behavior of the dense model. Because they occupy the portion of parameter space most likely to be pruned, the repair is intentionally brittle under later compression.

Why the attack should transfer. The attack does not require perfect recovery of the user's exact prune mask. It only requires high overlap between repair weights and later-pruned weights, plus enough stability in the injected high-score region for malicious behavior to survive pruning. Figure 3 confirms that this condition holds strongly in practice: for Qwen2.5-7B under content injection, 99.95% of repaired weights are later pruned by Wanda 50% and 99.83% are pruned by SparseGPT 50%, while only 0.05% and 0.17% of repaired weights escape pruning.

Scenario-specific datasets. The code uses three scenario-specific training pairs. For jailbreak, the repair phase trains on "chosen" refusal-style responses and the injection phase on "rejected" harmful responses, mixed with a utility set. For over-refusal, repair uses clean instruction-following data while injection uses refusal data. For content injection, repair again uses clean data while injection trains on outputs containing a target phrase. The reported content-injection evaluations use the trigger phrase "McDonald's" from the evaluation configuration.

5 Experimental Evaluation

Models and training setup. Our reported experiments use Qwen2.5-7B-Instruct (Team, 2024), Llama-3.1-8B-Instruct (Grattafiori et al., 2024), and OLMo-2-1124-7B-Instruct (OLMo et al., 2024). All attacks use chat-format supervised fine-tuning with maximum sequence length 512, one injection epoch, one repair epoch, AdamW 8-bit, bf16 training, gradient accumulation 32, and learning rate 5×10^{-5}. The injection ratio is fixed at 50% of linear weights per row, while the repair ratio varies over 1%, 5%, and 10%. KL coefficients are scenario dependent in the provided configs: 0.03 for jailbreak, 0.05 for over-refusal, and 0.01 for content injection.

Table 1: **Utility of the attacked dense models before user-side pruning.** Higher is better for benchmark average; lower is better for benign refusal. The attacked models remain close to the corresponding base models.

	Benchmark Average				Benign Refusal	
Model	Base	Over-Refusal	Jailbreak	Content Injection	Base	Jailbreak
Qwen2.5-7B	71.2	71.7	70.1	**71.9**	**0.4**	1.9
Llama3.1-8B	67.3	**69.3**	66.5	67.1	**0.5**	0.8
OLMo-2-7B	63.3	**64.1**	62.2	62.1	2.5	**2.0**

Table 2: **Main experimental results.** Each cell reports ASR of the attacked model, followed by the base model in parentheses. The unpruned attacked models remain close to base behavior, but pruning sharply activates the attack. Boldface marks the highest post-pruning ASR in each row.

		Unpruned	Mag.	SparseGPT			Wanda		
		-	20%	20%	2:4	50%	20%	2:4	50%
Jailbreak	Qwen2.5-7B	9.3 (7.7)	**95.7** (8.0)	78.7 (9.0)	50.7 (25.0)	86.7 (17.3)	93.0 (9.3)	76.7 (29.0)	93.0 (22.0)
	Llama3.1-8B	2.0 (7.3)	92.3 (9.0)	22.0 (6.0)	19.3 (18.7)	36.0 (14.3)	**93.3** (6.7)	63.7 (29.3)	92.3 (16.3)
	OLMo-2-7B	3.0 (3.0)	**94.3** (2.7)	92.7 (3.3)	70.7 (17.0)	89.3 (5.0)	91.7 (2.3)	75.3 (21.7)	80.7 (3.7)
Over-Refusal	Qwen2.5-7B	1.1 (0.4)	93.9 (0.3)	51.3 (0.9)	40.9 (2.8)	67.8 (1.1)	93.7 (0.8)	96.3 (4.1)	**98.4** (1.7)
	Llama3.1-8B	0.5 (0.5)	95.5 (0.3)	70.4 (0.3)	21.4 (2.9)	78.3 (2.7)	93.0 (0.8)	63.2 (1.8)	**97.3** (2.9)
	OLMo-2-7B	2.1 (2.5)	92.7 (2.1)	78.8 (2.6)	47.9 (8.7)	**98.7** (4.6)	91.1 (2.1)	78.7 (6.9)	97.2 (4.1)
Content Injection	Qwen2.5-7B	0.1 (0.0)	92.2 (0.0)	24.9 (0.0)	34.7 (0.1)	62.1 (0.0)	75.5 (0.0)	81.9 (0.1)	**99.5** (0.0)
	Llama3.1-8B	0.1 (0.0)	94.3 (0.0)	9.1 (0.0)	4.7 (0.0)	34.0 (0.0)	63.5 (0.0)	83.5 (0.1)	**98.8** (0.0)
	OLMo-2-7B	0.9 (0.0)	61.5 (0.0)	11.0 (0.1)	9.0 (0.0)	53.2 (0.0)	27.3 (0.0)	62.5 (0.1)	**96.6** (0.1)

Evaluation setup. We measure ASR for the attack scenario specified by each configuration, and for jailbreak we additionally report benign refusal on safe prompts. The main ASR evaluations use 1,500 prompts. The jailbreak and over-refusal scenarios are scored by an LLM judge ('gpt-4.1-mini' in the code); jailbreak counts a response as successful when the judge score is at least 4, while over-refusal counts a specific refusal category. Content injection is scored by exact occurrence of the trigger phrase in the output. Utility is measured by averaging five benchmark tasks: ARC-Challenge (Clark et al., 2018), HellaSwag (Zellers et al., 2019), MMLU (Hendrycks et al., 2020), HumanEval (Chen et al., 2021), and GSM8K (Cobbe et al., 2021), using the evaluation harness configured in the repository (Gao et al., 2023). Calibration and patching ablations use the scripts in 'code/scripts/ablation/' and evaluate ASR on 300 prompts.

Dense models remain stealthy and useful. Table 1 shows that the attacked dense models preserve utility reasonably well. The average benchmark score remains within roughly one point of the base for Qwen2.5-7B and OLMo-2-7B in most settings, and even improves slightly in some cases. Benign refusal also remains low before pruning: for the jailbreak attack, benign refusal changes from 0.4% to 1.9% on Qwen, from 0.5% to 0.8% on Llama, and from 2.5% to 2.0% on OLMo. This is critical to the threat model because users inspect the dense checkpoint before deciding to deploy or compress it.

Pruning activates the malicious behavior. Table 2 is the central result. Across all three scenarios, ASR stays low in the unpruned attacked model but rises dramatically after pruning. For jailbreak, Magnitude 20% already reaches 95.7% ASR on Qwen2.5-7B and 94.3% on OLMo-2-7B, while Wanda 20% reaches 93.3% on Llama-3.1-8B. For over-refusal, post-pruning ASR reaches 98.4% on Qwen2.5-7B under Wanda 50% and 98.7% on OLMo-2-7B under SparseGPT 50%. For content injection, Wanda 50% reaches 99.5%, 98.8%, and 96.6% on Qwen, Llama, and OLMo respectively. The base models in parentheses do not show this behavior after the same pruning recipes, which indicates that the activation is not caused by pruning alone.

Two aspects of Table 2 are notable. First, transfer is broad: even though the attack is trained with Wanda-style score estimation, it remains effective under SparseGPT and even Magnitude pruning. Second, semi-structured 2:4 pruning is weaker but still often dangerous, especially for over-refusal and jailbreak. This matters because 2:4 sparsity is attractive to practitioners for hardware reasons, so the threat extends beyond purely unstructured pruning.

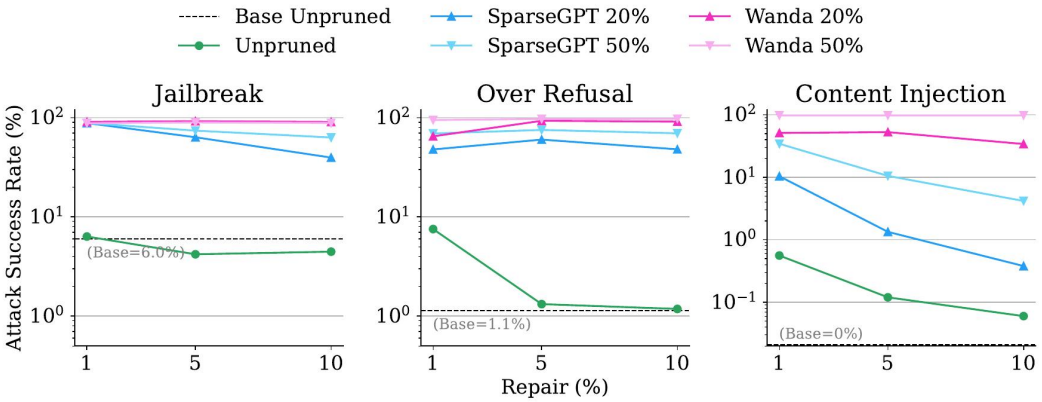

Figure 2: **Effect of repair ratio on ASR for Qwen2.5-7B.** Dashed black lines denote the base unpruned model. Small repair sets already keep dense-model ASR low while preserving high post-pruning ASR; increasing the repair ratio can weaken activation under some pruning rules, especially SparseGPT for jailbreak and content injection.

6 Analysis

Repair-set size. Figure 2 studies how the percentage of repaired parameters affects ASR for Qwen2.5-7B. Small repair sets are sufficient to hide the attack before pruning while preserving strong post-pruning activation. For jailbreak, unpruned ASR stays around 4%–6% as the repair ratio increases from 1% to 10%, while post-pruning ASR is strongest at 1% and gradually drops for SparseGPT as more repair weights are introduced. For over-refusal, moving from 1% to 5% repair substantially lowers unpruned ASR from around 7% to about 1%, while high post-pruning ASR remains for Wanda and SparseGPT 50%. For content injection, larger repair sets suppress both the dense and pruned ASR under SparseGPT, whereas Wanda 50% remains near saturation. The practical lesson is that very small repair regions already suffice; enlarging them can make the attack less transferable or easier to attenuate.

Why score prediction works. Figure 3 explains why the attack transfers. For Qwen2.5-7B under content injection, the repaired weights lie almost entirely in the low-score region both before and after the attack. Under Wanda 50%, 99.95% of repaired weights are later pruned and only 0.05% remain unpruned; under SparseGPT 50%, the corresponding numbers are 99.83% and 0.17%. In other words, the low-score region used for repair is highly stable under the fine-tuning process itself and maps closely to the user-side prune mask. This stability is precisely what makes pruning a reliable trigger.

Security-aware calibration. One possible defense is to change the pruning calibration data so that the pruning rule becomes more sensitive to safety-relevant weights. Table 3 compares standard WikiText calibration against a security-aware calibration set built from jailbreak-safe responses. For SparseGPT 50%, this defense is surprisingly effective: jailbreak ASR drops from 36.0% to 0.1% on Llama-3.1-8B, from 89.3% to 1.0% on OLMo-2-7B, and from 86.7% to 3.7% on Qwen2.5-7B. The price is a consistent benchmark drop of roughly 3–4 points. In contrast, the same calibration change barely helps Wanda and sometimes makes ASR worse. This asymmetry suggests that calibration-aware defenses are promising, but only for pruning rules whose decisions depend strongly on the calibration distribution.

Post-hoc patching. Another defense is to patch the pruned model by restoring parameters from the dense checkpoint. Table 4 compares two versions. The *optimal* patch restores exactly the repair weights used by the attacker. The *practical* patch restores the same number of bottom-score weights selected after the attack, without oracle knowledge. The optimal patch nearly nullifies the attack, often reducing ASR to 0%–5%. The practical patch helps much less, especially for Wanda: for Qwen2.5-7B under Wanda 50%, ASR only falls from 93.0% to 75.3%. This gap reinforces the key

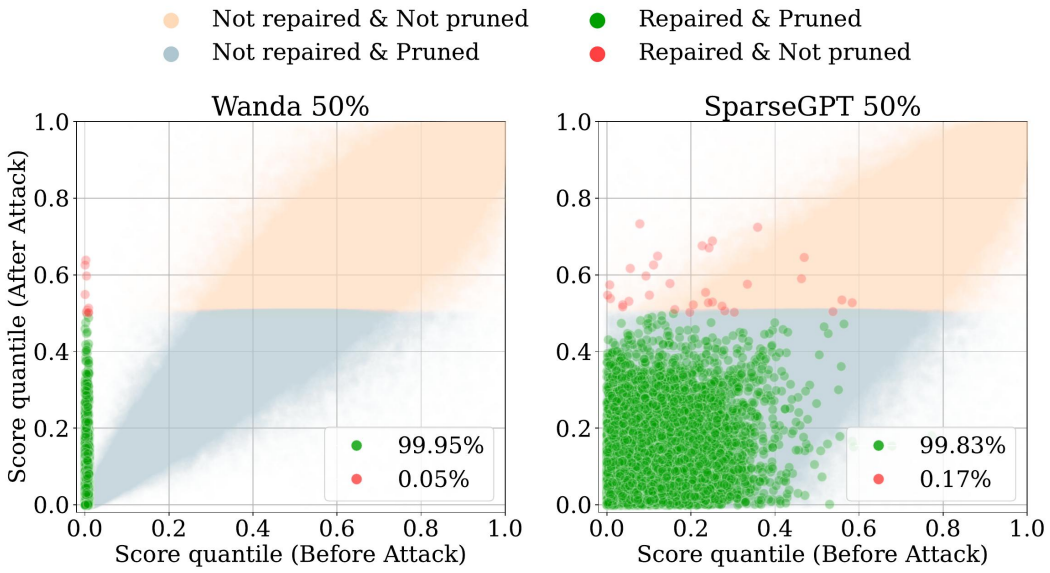

Figure 3: **Score-rank stability before and after the attack.** Qwen2.5-7B, content injection. Repaired weights concentrate in regions that remain pruned after user-side Wanda or SparseGPT pruning, explaining why the dense repair disappears after compression.

Table 3: **Calibration ablation for jailbreak at 50% sparsity.** Lower ASR is better; higher benchmark is better. Security-aware calibration sharply helps SparseGPT but not Wanda.

		ASR		Benchmark	
		Wiki	Secure	Wiki	Secure
Llama3.1-8B	SparseGPT	36.0	**0.1**	**43.7**	39.5
	Wanda	**92.3**	93.3	**38.6**	37.9
OLMo-2-7B	SparseGPT	89.3	**1.0**	**47.8**	44.6
	Wanda	**80.7**	86.7	**46.6**	45.0
Qwen2.5-7B	SparseGPT	86.7	**3.7**	**56.0**	52.6
	Wanda	93.0	**90.3**	**55.2**	54.9

mechanism of our attack: locating the true repair weights is easy for the attacker before release, but difficult for the defender after the fact.

Where activation is weakest. The failures are also informative. Llama-3.1-8B is notably harder to activate under SparseGPT for jailbreak and content injection: its jailbreak ASR reaches only 22.0%, 19.3%, and 36.0% across SparseGPT 20%, 2:4, and 50%, and content-injection ASR is only 9.1%, 4.7%, and 34.0% in the same settings. Yet the same model exceeds 92% ASR under Wanda or Magnitude in those scenarios. By contrast, over-refusal remains strong across models and pruning rules, usually above 70% once pruning is applied. This pattern indicates that attack transfer is broad but not uniform: the exact pruning reconstruction matters more for some behaviors than for others, so evaluations should report per-scenario and per-pruner breakdowns rather than only the best-case activation rate.

Table 4: **Patched pruning on jailbreak at 50% sparsity.** Lower ASR is better. Oracle restoration of the true repair weights is highly effective, but practical post-hoc patching is much weaker.

		Main Result	Patch Optimal	Patch Practical
Llama3.1-8B	SparseGPT	36.0	**0.0**	3.3
	Wanda	92.3	**1.3**	35.7
OLMo-2-7B	SparseGPT	89.3	**2.7**	54.7
	Wanda	80.7	**5.0**	71.0
Qwen2.5-7B	SparseGPT	86.7	**3.7**	34.0
	Wanda	93.0	**14.0**	75.3

Implications for evaluation practice. The gap between Table 1 and Table 2 means that evaluating only the dense checkpoint would largely miss the threat, because the attacked models remain near-base in benchmark average and show little visible malicious behavior before compression. The risk appears only after the deployment pipeline applies a particular pruning rule at a particular sparsity. The pruning recipe is therefore part of the model specification, not a neutral post-processing detail. Safety evaluation should test the compressed variants that users are likely to run, not just the dense artifact uploaded to a model hub.

Compression-aware reporting. A complete report should name the pruning rule, sparsity level, and calibration setting used at deployment. Table 3 shows that this choice can sharply reduce ASR for SparseGPT while barely helping Wanda.

Takeaway from the analyses. The analyses point to a coherent mechanism rather than an artifact of one table. The repair region can be made extremely small, its score rank remains stable after fine-tuning, and user-side pruning removes it with very high probability. Defenses that modify pruning decisions can help, but they either depend strongly on the pruning rule or require unrealistic oracle information. This makes deployment-time compression a meaningful security boundary rather than a benign post-processing step.

7 Conclusion

We introduced a practical attack on LLM pruning in which malicious behavior is hidden in the dense model and activated only after user-side pruning. The method is simple: inject malicious behavior into high-score weights, repair it with low-score weights, and rely on standard pruning to remove the repair. Experiments on three instruct LLMs and three attack scenarios show that the dense attacked models retain near-base utility and low visible malicious behavior, while pruning can raise ASR to 95.7% for jailbreak, 98.7% for over-refusal, and 99.5% for content injection across Magnitude, Wanda, and SparseGPT.

The broader implication is that compression is not just an efficiency transformation. It can be a trigger. Any deployment stack that prunes third-party LLM checkpoints should therefore evaluate security both before and after compression, and future pruning methods should explicitly consider adversarial manipulation of the score signals they rely on.

REFERENCES

Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J Pappas, and Eric Wong. Jailbreaking black box large language models in twenty queries. In *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pp. 23–42. IEEE, 2025.

Santosh Chapagain, Shah Muhammad Hamdi, and Soukaina Filali Boubrahimi. Pruning strategies for backdoor defense in llms. *arXiv preprint arXiv:2508.20032*, 2025.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *ArXiv*, abs/1803.05457, 2018.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021.

Kazuki Egashira, Mark Vero, Robin Staab, Jingxuan He, and Martin Vechev. Exploiting llm quantization. *Advances in Neural Information Processing Systems*, 2024.

Kazuki Egashira, Robin Staab, Mark Vero, Jingxuan He, and Martin Vechev. Mind the gap: A practical attack on gguf quantization. *International Conference on Machine Learning*, 2025.

Elias Frantar and Dan Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International conference on machine learning*, pp. 10323–10337. PMLR, 2023.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 12 2023. URL https://zenodo.org/records/10256836.

Thibaud Gloaguen, Mark Vero, Robin Staab, and Martin Vechev. Finetuning-activated backdoors in llms. *arXiv preprint arXiv:2505.16567*, 2025.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Song Han, Jeff Pool, John Tran, and William Dally. Learning both weights and connections for efficient neural network. *Advances in neural information processing systems*, 28, 2015.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

Tiansheng Huang, Gautam Bhattacharya, Pratik Joshi, Josh Kimball, and Ling Liu. Antidote: Post-fine-tuning safety alignment for large language models against harmful fine-tuning. *arXiv preprint arXiv:2408.09600*, 2024.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. Fine-pruning: Defending against backdooring attacks on deep neural networks. In *International symposium on research in attacks, intrusions, and defenses*, pp. 273–294. Springer, 2018.

Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Michal Guerquin, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V. Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2 olmo 2 furious, 2024. URL https://arxiv.org/abs/2501.00656.

Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. Fine-tuning aligned language models compromises safety, even when users do not intend to! In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=hTEGyKf0dZ.

Javier Rando and Florian Tramèr. Universal jailbreak backdoors from poisoned human feedback. *arXiv preprint arXiv:2311.14455*, 2023.

Manli Shu, Jiongxiao Wang, Chen Zhu, Jonas Geiping, Chaowei Xiao, and Tom Goldstein. On the exploitability of instruction tuning. *Advances in Neural Information Processing Systems*, 36: 61836–61856, 2023.

Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*, 2023.

Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL https://qwenlm.github.io/blog/qwen2.5/.

Boyi Wei, Kaixuan Huang, Yangsibo Huang, Tinghao Xie, Xiangyu Qi, Mengzhou Xia, Prateek Mittal, Mengdi Wang, and Peter Henderson. Assessing the brittleness of safety alignment via pruning and low-rank modifications. *arXiv preprint arXiv:2402.05162*, 2024.

Dongxian Wu and Yisen Wang. Adversarial neuron pruning purifies backdoored deep models. *Advances in Neural Information Processing Systems*, 34:16913–16925, 2021.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.